用户手册

Robot SDK — Python 用户手册

Windows 与 Linux x86_64 | Python 3.9–3.13

SDK 1.5.0 · 文档版本：V1.4

安全 — 请在空旷场地、低速下验证每一次运动。软件限位只是附加保护，不能代替安装检查、急停流程或应用层联锁。

文档修订记录

版本	日期	说明
V1.0	2026-07-23	首次分别提供 C++ 与 Python 用户手册（Spark2 SDK）。
V1.1	2026-08-06	应用程序使用 C++17。Windows 工具说明改为 Visual Studio 2022 之后、MSVC v143。Python 3.10–3.14。控制优先级。路径长度规则。t 的单位为秒。
V1.2	2026-08-10	改为发行包安装。控制优先级（物理按钮高于标准客户端；同一优先级需等待）。当时将 v=0 描述为最大速度。夹爪位置为 0–1。限位越界。状态反馈更名。
V1.3	2026-09-24	对齐 Robot SDK 1.3.0。客户端为 Robot；会话为 connect / disconnect（Python 的 stop 仍是 disconnect 的别名）。在线客户端不接收配置文件路径。使用 RtConnectionInfo 或枚举到的机器人连接。关节向量长度跟随运行时关节数。v=0 保持当前速度比例，而不是最大速度。moveVel / move_vel 的加速度默认值为 10。夹爪指令长度等于夹爪关节数。增加等待空闲、事件、诊断和网络配置。运动、反馈和配置针对 0 号机械臂。同一手册覆盖 Windows x64 MSVC 2019 包（Python 3.9–3.12）和 Linux x86_64 GCC 包（Python 3.9–3.13）。
V1.4	2026-11-11	对齐 Robot SDK 1.5.0（Linux x86_64 包，2026-11-04，GCC 9.4）。C++ 与 Python 提供单臂句柄和同步双臂句柄。关节点位和回零使用 MotionOptions；这些调用的数字参数 v、t 已弃用。速度模式要求 v 在 (0, 100] 内，v = 0 会被拒绝。t > 0 时忽略 v，并使用全部已配置限位。路径自动选择五次多项式路径平滑。关节等待需到达 1 度以内，并报告被拒绝的规划。新增折叠、运动停止、保护性停止、夹爪等待、停止录制和保存轨迹。连接状态取代连接质量统计。故障在机器人客户端上清除。可以在事件回调中调用 disconnect()。

文档范围

本手册说明如何安装 Robot SDK 1.5.0（Python）并控制机器人。

概览

项目	支持范围
SDK	Robot SDK 1.5.0
Linux 包	2026-11-04。Linux x86_64，GCC 9.4。Python wheel 覆盖 3.9–3.13。含 examples/cpp 与 examples/python。
Windows 包	Windows 10 / 11 x64，MSVC 2019 工具集 v142。Python wheel 覆盖 3.9–3.12。含 examples/cpp、examples/python 与 examples/csharp。
语言	Linux 上为 Python 3.9–3.13；Windows 上为 Python 3.9–3.12
构建 / 安装	用 pip 安装该操作系统对应的 bindings/python 中的 wheel 安装包
文档版本	V1.4（2026-11-11）

目录

- 文档修订记录
- 1 安装与首次运行
- 2 运行流程与约定
- 3 运动接口
- 4 交互、反馈与状态
- 5 配置与安全限位
- 6 运动学与坐标系变换
- 附录 A 接口索引与类型

术语

术语	含义
关节空间	connect 之后，指令或反馈的长度等于 joint_count() / jointCount()。位置单位为度，速度单位为度/秒。
0 号机械臂	编号为 0 的机械臂。Robot 上不指定臂号的调用都作用在这条臂上。其他臂使用 arm(index)。臂 0 和臂 1 一起运动时使用双臂句柄。
末端位姿	机器人末端坐标系的位姿。
工具位姿	已配置工具坐标系的位姿。姿态与末端位姿相同，平移包含当前工具偏移。
工具偏移	工具坐标系相对末端坐标系的平移 (x, y, z)。不是完整位姿。
点到点	控制器规划一条到达目标位姿的运动。
直线	笛卡尔直线运动。指令所指定的坐标系沿直线移动。
RobotResult	指令成功或失败。非阻塞运动返回成功只表示指令已提交，不表示机械臂已到达，也不表示规划已被接受。
MotionOptions	关节点位和回零的显式计时方式，分为限速和定时。优先于这些调用上的数字参数 v、t。

1. 安装与首次运行

1.1 Python 环境

平台	要求
Windows x64	Windows 10 / 11。Python 3.9、3.10、3.11 或 3.12。wheel 标签为 win_amd64，cpXY 须与该解释器一致。
Linux x86_64	Linux x86_64。Python 3.9、3.10、3.11、3.12 或 3.13。wheel 标签为 linux_x86_64。本包中的 1.5.0 库使用 GCC 9.4 构建。
机器人	机器人与电脑在同一以太网，或使用机器人 IP 地址连接。

1.2 包目录

请使用与当前操作系统匹配的发行包，并保持目录结构不变。

```
<prefix>/
  bindings/python/     本操作系统的 Python wheel 安装包
  docs/                接口说明和本手册
  examples/python/     示例和 utils.py
  examples/cpp/        C++ 示例
  include/             C++ 头文件
  lib/                 库和 CMake 包
```

Windows 包另有 bin/（robot_sdk.dll、robot_sdk_c.dll）和 examples/csharp/。Linux 库为 lib/librobot_sdk.so 与 lib/librobot_sdk_c.so。

1.3 快速开始

安装与当前 Python 解释器和操作系统匹配的 wheel 安装包。

```
# Windows, Python 3.12
pip install .\bindings\python\robot_sdk-1.5.0-cp312-cp312-win_amd64.whl
# Linux, Python 3.12
pip install ./bindings/python/robot_sdk-1.5.0-cp312-cp312-linux_x86_64.whl
python -c "from robot_sdk import Robot; print('Robot SDK import ok')"
```

机器人默认 IP 地址为 192.168.10.147。用网线连接电脑和机器人。将电脑以太网地址设为 192.168.10.*，子网掩码 255.255.255.0。末位为 1–254，且不能是 147。

在包根目录运行示例。不传 IP 时，示例连接局域网中发现的第一台机器人。传入机器人 IP 可指定一台。示例还包括 fold_arm.py、teach_and_playback.py、dual_arm_move_point.py 和 dual_arm_move_path.py。

```
# Windows
cd .\examples\python
python .\move_point.py
python .\move_point.py 192.168.10.147
# Linux
cd examples/python
python3 move_point.py
python3 move_point.py 192.168.10.147
```

最小程序：

```
from robot_sdk import Robot, RtConnectionInfo, enumerate_robots

robot = Robot()
result = robot.connect(RtConnectionInfo())  # auto-discover
if not result.ok:
    raise SystemExit(result.message)
mask = [True] * robot.joint_count()
robot.enable_arm_joint(mask).raise_for_error()
# move_pos, move_tool_point, go_home, ...
robot.disconnect()

robot.connect(RtConnectionInfo.to("192.168.10.147"))
robots = enumerate_robots()
if robots:
    robot.connect(robots[0])
```

enumerate_robots() 是模块函数，不会保持会话。将返回的一台机器人传给 connect()。

1.4 首次运行检查

- 确认机器人、控制器和安全系统已完成正常启动检查。
- 先连接，再读取 joint_count()，并使每条关节指令的长度与之相同。
- 使机械臂避开人员、工装、线缆和行程末端。
- 从低速开始，例如 MotionOptions.speed_limited(20)，确认指令坐标系后再提高速度。
- 应用程序退出时调用 disconnect()。

2. 运行流程与约定

2.1 建议的控制顺序

步骤	动作	原因
1	Robot()	创建客户端。
2	connect(info 或 robot)	开始会话。服务器 IP 为空时自动发现；已知 IP 或已枚举的机器人会跳过发现。
3	enable_arm_joint(mask)	使能 0 号机械臂。掩码长度必须等于连接后报告的关节数。
4	先读状态，再发运动	提高速度前确认机械臂空闲且已使能。
5	disconnect()	结束会话并释放控制权。不要依赖进程退出。

2.2 客户端生命周期

调用	参数和结果	用途 / 前提
Robot()	无参数。	创建一次客户端，然后连接。
enumerate_robots()	模块函数。返回网络上发现的机器人，不保持会话。	可能有多台机器人时应先枚举，再连接其中一台。
connect(info 或 robot)	RtConnectionInfo，或一台已发现的机器人。返回 RobotResult。	运动、示教、回放、写配置或读取实时反馈之前必须连接。正常连接会等待新的空闲状态。
disconnect() / stop()	结束会话。返回 RobotResult。stop() 与 disconnect() 相同。	正常退出时调用。允许在事件回调中调用；套接字清理在下一次应用线程调用时完成。
enable_arm_joint(mask)	布尔掩码，长度等于关节数。true 表示使能该关节。	使能 0 号机械臂。两臂一起使能时使用 dual_arm().enable_arm_joint(mask0, mask1)。
arm(index)	单臂的非拥有句柄。	重新连接后需要重新获取。连接后会按拓扑检查索引。
dual_arm()	臂 0 与臂 1 的同步句柄。	未连接或机械臂少于两条时抛出异常。
set_arm_smoothing_method (method)	平滑策略。	在 Robot 上调用时作用于每一条臂。在 arm(i) 上调用时只作用于该臂。路径调用会自行选择五次多项式路径平滑。
release_control()	无运动参数。返回 RobotResult。	释放 SDK 控制权。之后的指令会失败，直到再次连接。
stop_motion()	可选 wait_for_command_ack，默认 false。保持当前实测位姿。	点动或运动过程中的软件停止。不会进入 ERROR，也不能代替硬件急停。
trigger_safeguard_stop()	无参数。返回 RobotResult。	保护性停止，与面板中止同一类。该停止会锁存，直到机器人报告 ERROR。需要已连接，不需要持有控制权。

故障 — 用 get_status() 读取诊断。在机器人客户端清除故障；若会话已不可用，请重新连接。

2.3 控制优先级

实时控制器决定哪个客户端可以指挥机器人。物理示教按钮、Robot SDK、面板和网页可以同时存在。高优先级客户端持有控制权时，低优先级客户端会被阻止。同一优先级的客户端不会互相抢占：同级客户端要等到当前控制者结束并释放控制权。

优先级（从高到低）	客户端（示例）	含义
物理按钮	物理示教按钮	最高的操作客户端。它持有控制权时，标准客户端失去独占控制。
标准	Robot SDK；面板；网页	同一优先级。标准客户端需等待当前标准控制者释放控制权。

优先级 — 若其他客户端取得控制权，运动调用以 CONTROL_LOST 失败。等该客户端释放控制权后再连接。

2.4 默认值与单位

连接后，每条臂的向量长度必须等于该臂的关节数。关节位置单位为度，关节速度单位为度/秒。夹爪位置指令使用面板单位 [0, 1]（0 为张开，1 为闭合），不是配置器中的软限位或硬限位。速度 v 和加速度 a 为百分比。默认 v 为 50。速度指令的默认加速度为 10。时间 t 的单位为秒，默认 t 为 0。

Robot 上的便捷接口仍针对 0 号机械臂。arm_count() 是机械臂数量，gripper_count() 是夹爪数量。整机空闲等待覆盖每一条臂。只动夹爪时机器人可能仍为 Idle，因此应等待夹爪反馈。

2.5 MotionOptions

Robot、单臂句柄和双臂句柄上的关节点位与回零使用 MotionOptions。speed_limited 的百分比在 (0, 100] 内，时长由控制器决定。timed 的时长必须大于 0，并使用全部已配置限位。定时同时又降低速度上限会返回 Unsupported。控制器可以延长所请求的时长，但不会缩短。

Robot、Arm 和 DualArm 上 move_pos、move_pos_and_wait、go_home、go_home_and_wait 的数字参数 (v, t) 已弃用。它们仍可调用，并产生 DeprecationWarning。t = 0 为速度模式，要求 v 在 (0, 100] 内。v = 0 会被拒绝，不会保持上一次的速度百分比。t > 0 时忽略 v。

路径、笛卡尔、速度、夹爪和折叠调用不在此次弃用范围内，仍使用数字 v 和 t。路径上的 t[i] 是该段的时长，不是整条路径的时长。时间列表为空时，每一段都使用速度模式。速度列表为空时，速度段使用 50%。非空的计时列表必须与路点数量一致。允许速度段和定时段混合。

```
from robot_sdk import MotionOptions

# Preferred. Speed percent in (0, 100], duration chosen by the controller.
robot.go_home(options=MotionOptions.speed_limited(20)).raise_for_error()
robot.move_pos_and_wait(q, MotionOptions.timed(5.0), timeout_sec=30)

# Deprecated numeric form. Same rule: t = 0 uses v; t > 0 ignores v.
# robot.go_home(v=20, t=0.0)
```

2.6 单臂与双臂句柄

从 robot_sdk 导入 Arm 和 DualArm。机器人未连接或机械臂少于两条时，dual_arm() 抛出 RobotError。句柄会保持其 Robot 存活。重新连接后需要重新获取句柄。

同一次双臂调用中的两条臂共用同一计时。每个关节向量必须匹配该臂的关节数。路径路点数量必须一致。先调用臂 0 再调用臂 1 不会形成同步运动。Robot 上的 set_arm_smoothing_method 仍作用于每一条臂。两臂需要一起回零时，使用双臂回零调用。

```
left = robot.arm(0)
right = robot.arm(1)
both = robot.dual_arm()
both.enable_arm_joint(
    left.all_enabled_mask(), right.all_enabled_mask()
).raise_for_error()
both.move_pos_and_wait(
    q0, q1, MotionOptions.speed_limited(30)
).raise_for_error()
```

2.7 完成、错误与线程

非阻塞运动调用成功只表示指令已在本机提交。不表示控制器接受了规划，也不表示机械臂已到达。下一步依赖到达时，使用 and-wait 调用。超时不会取消运动。若等待到期后必须保持当前位置，请调用 stop_motion()。

关节位置、回零和折叠的等待，只有在机械臂进入目标 1 度以内，且连续的新空闲样本属于本条指令时才成功。双臂关节等待会检查两条臂。若较新的控制器把目标限制在限位内，等待使用限制后的目标。无法确认该目标时，等待失败。较旧的控制器仍使用所请求的目标，未到达时可能超时。

在更新后的控制器上，即使机器人状态为 Idle，被拒绝的规划也会由等待接口报告。非法目标、实测起点超限或轨迹越过限位返回命令无效。触及运动边界而停止时，返回运动被中断。消息包含指令序号，以及已知时从 0 开始的臂和关节索引。这些是运动错误，不是硬件故障。不发送拒绝标志的较旧控制器仍按原来的超时方式处理。

路径调用会等待配置应答和指令应答，然后返回。它不会等到路径运动结束。配置不一致时，运动不会发出。开始等待之前请检查结果。

结果	处理方式
非阻塞运动成功	指令已提交。确认到达前请等待或读取状态。
参数无效	修正向量长度、索引、速度或时长。原样重发仍会失败。
命令无效	目标、实测起点或轨迹被拒绝。请查看消息。这本身不是硬件故障。
运动被中断	触及运动边界而停止，或等待被恢复流程或示教按钮打断。
超时	等待已到期。在主动停止之前，运动仍然有效。
失去控制	其他客户端持有控制权。等它释放后再连接。
不支持	该调用不可用。带速度上限的定时运动，或较旧控制器上的轨迹导出，会返回此项。

同一个 Robot 的连接、运动和查询请在一个线程中调用。状态回调运行在 SDK 事件线程上，并且按 Robot 串行。运动完成回调运行在调用等待的线程上。回调应尽量短。在回调中调用 disconnect() 会立即关闭会话。不要在回调中调用 connect() 或阻塞运动接口。请检查 result.ok，或调用 raise_for_error()。dual_arm() 可能抛出 RobotError。

回调 — 不要在回调中调用 connect、阻塞运动或阻塞等待。允许调用 disconnect。

3. 运动接口

除非另行说明，Robot 上的调用返回 RobotResult，并指令 0 号机械臂。单臂使用 arm(index) 上的相同运动方法。关节列表必须匹配该臂的关节数。

3.1 关节空间运动

调用	参数	用途
move_pos(arm_pos, options)	arm_pos：度，长度等于关节数。options：MotionOptions。	0 号机械臂的首选关节点位运动。
move_pos(arm_pos, v=50, t=0)	已弃用的数字参数形式。	t = 0 时要求 v 在 (0, 100] 内。t > 0 时忽略 v。
move_pos_path(path, v_path, t_path)	关节路点，单位为度。v 和 t 按路点给出。	执行关节路径。该调用选择五次多项式路径平滑，并等待配置应答和指令应答。
move_vel(arm_vel, a=10, t=0)	arm_vel：度/秒。a：加速度百分比，默认 10。t：秒，默认 0。	指令关节速度。未弃用，仍使用数字计时。
go_home(options)	MotionOptions。	回零的首选方式。请确认路径畅通。
go_home(v=50, t=0)	已弃用的数字参数形式。	速度和时间规则与 move_pos 相同。
fold_arm(v=50, t=0)	数字 v 和 t。不是 MotionOptions 调用。	将 0 号机械臂运动到内置折叠姿态。

3.2 等待运动结束

调用	用途
wait_until_idle(timeout_sec=30, poll_interval_sec=0.1)	阻塞到整机空闲、报告拒绝或超时。
move_and_wait(move, timeout_sec=30, ...)	发送一个运动可调用对象，然后等待。
move_pos_and_wait(arm_pos, options, ...)	关节点位运动，然后确认到达 1 度以内且为空闲。
go_home_and_wait(options, ...)	回零，然后做同样的到达检查。
fold_arm_and_wait(v=50, t=0, ...)	折叠，然后做同样的到达检查。使用数字 v 和 t。
move_gripper_pos_and_wait(pos, ...)	夹爪指令，然后等待夹爪反馈。只动夹爪时不要使用空闲等待。

运动路径 — 接口调用成功并不能证明扫过的路径是安全的。请验证工装间隙、工具外形、线缆走向，以及整个任务的奇异点和关节限位。

3.3 笛卡尔运动：末端坐标系与工具坐标系

目标定义在末端坐标系时使用末端指令。目标定义在当前工具坐标系时使用工具指令，以便应用已配置的工具偏移。这些调用仍使用数字 v 和 t。

目标坐标系	点到点	直线	路径形式	适用场合
末端	move_ee_point	move_ee_line	move_ee_point_path / move_ee_line_path	位姿定义在末端坐标系。
工具	move_tool_point	move_tool_line	move_tool_point_path / move_tool_line_path	任务定义在已配置的工具坐标系。

3.4 笛卡尔参数

参数	含义	使用说明
pose	所选坐标系中的目标笛卡尔位姿。	发运动前确认坐标系。
path	所选坐标系中按顺序排列的笛卡尔路点。	整条路径使用同一个坐标系。非空的速度或时间列表必须与路点数量一致。
v / v 路径	单个目标或每个路点的速度百分比。	单目标默认 v 为 50。v = 100 为满量程。哪些调用拒绝 v = 0，见 2.5 节。

参数	含义	使用说明
t / t 路径	单个目标或每个路点的时间，单位为秒。	默认 t 为 0。位置类定时运动中，控制器可以延长时长，但不会缩短。仅给出时间的路径，其速度段按 50% 速度执行。

坐标系 — 不要混用末端位姿和工具位姿。工具偏移改变后，请重新确认任务位姿、接近路径和已保存的路点。

3.5 同步双臂运动

在 arm_count() 至少为 2 的机器人上，connect 之后用 dual_arm() 获取句柄。

调用	用途
enable_arm_joint(mask0, mask1)	在一次配置中使能两条臂。
move_pos / move_pos_and_wait	共用 MotionOptions。每个向量匹配对应的臂。
move_pos_path / move_pos_path_and_wait	共用数字 v 和 t 路径。路点数量一致。
move_ee_* / move_tool_*	点位、直线和路径，以及对应的等待调用。使用数字 v 和 t。
move_vel / move_vel_and_wait	两条臂的关节速度。加速度百分比，默认 10。
go_home / go_home_and_wait	共用 MotionOptions。两条臂一起回零。

参见 examples/python/dual_arm_move_point.py 与 dual_arm_move_path.py。

4. 交互、反馈与状态

4.1 夹爪、示教与回放

调用	参数 / 结果	用途 / 前提
move_gripper_pos(pos, v=50, t=0)	pos 的长度等于 gripper_joint_count()。取值为 [0, 1]。	指令夹爪关节。0 为张开，1 为闭合。
move_gripper_pos_and_wait(...)	同一条指令，然后等待夹爪反馈。	下一步需要夹爪完成时使用。空闲等待可能提前返回。
start_teach()	无额外参数。返回 RobotResult。	进入拖动示教并开始录制。停止之后再次开始会替换上一段录制。
stop_recording()	无额外参数。返回 RobotResult。	冻结录制并保持拖动。不写文件，也不是暂停或继续。
stop_teach()	无额外参数。返回 RobotResult。	结束示教并指令关节位置保持。确认只表示保持指令已被接受，不表示机械臂已经静止。
save_trajectory(local_path)	客户端 JSON 路径。返回 RobotResult。	导出已冻结的录制。只能在 stop_recording 或 stop_teach 之后调用。文件只在完整导出后被替换。
start_playback() / stop_playback() / reset_playback()	无额外参数。返回 RobotResult。	开始、停止或复位控制器内存中的录制回放。不会把 JSON 文件再加载回去。

Robot 上的示教和回放针对 0 号机械臂。arm(i)
导出该臂已冻结的采样。开始或停止录制由控制器在各臂之间共享；若两条臂都在拖动，仍需分别结束示教。JSON 的 schema 为 robot_sdk.teaching_trajectory，版本
1。关节采样单位是弧度，不是度，并且不含夹爪。不能从文件再导入。

仍在录制时保存轨迹返回运动被中断；没有录制数据时返回命令无效；控制器不支持导出时返回不支持；没有 TCP 时返回传输不可用。

```
robot.start_teach().raise_for_error()
robot.stop_recording().raise_for_error()   # drag remains
robot.stop_teach().raise_for_error()       # hold current joints
robot.save_trajectory("teach.json").raise_for_error()
robot.wait_until_idle().raise_for_error()
```

参见 examples/python/teach_and_playback.py。可选参数 --trajectory 会写入 JSON 文件。

4.2 状态反馈

Robot 上的反馈针对 0 号机械臂，连接后可用。arm(index) 上有相同的读取。关节位置单位为度，关节速度单位为度/秒。

调用	返回	用途
get_joint_state() / get_joint_vel() / get_joint_tor()	0 号机械臂的位置、速度或力矩。	监视或记录关节反馈。长度跟随关节数。
joint_positions()	仅 0 号机械臂的位置，单位为度。	不需要速度和力矩时使用更短的读取。
get_ee_pose() / get_tool_pose()	当前末端位姿或工具位姿。	读取与所发指令相同的坐标系。
get_arm_joint_enabled_mask()	逐关节使能掩码，长度等于关节数。	运动前确认关节已使能。
get_status()	SystemStatus：机器人状态、规划结果和诊断标志。	关节诊断按臂、再按关节索引。
print_status(status)	打印一个 SystemStatus。	调试时打印状态。

4.3 事件与连接状态

用 get_status() 读取诊断。连接是否正常见下面的会话存活字段。

调用	线程 / 结果	用途
set_on_state_changed(cb)	SDK 事件线程。上一状态、当前状态和状态数据。	机器人状态变化。回调应尽量短。
set_on_motion_done(cb)	调用等待的线程。状态和结果。	运动等待结束，包括被拒绝的结果。
clear_event_callbacks()	清除已注册的回调。	关闭回调时从应用线程调用。
is_connected()	会话尚未锁存接收超时则为真。	快速检查是否在线。读取状态不会运行看门狗。
get_connection_status()	connected、telemetry_timed_out、receive_age_ms、receive_timeout_ms（3000）。	超时会一直锁存，直到下一次成功连接。

回调 — 不要在回调中调用 connect 或阻塞运动接口。允许调用 disconnect，它会立即关闭会话。

4.4 网络配置

这些调用需要已建立的会话。更改有线地址会使当前会话断开；请用新地址重新连接。

调用	用途 / 注意
get_network_config()	读取有线和 Wi-Fi 设置。默认应答超时 5000 毫秒。返回结果和配置。Wi-Fi 密码会被掩码。
set_wifi(ssid, password, enable=True)	将机器人 Wi-Fi 连接到路由器。
set_eth_static(ip, netmask, gateway, dns="")	设置静态有线地址。链路会断开。请用新 IP 重新连接。
reset_network_config()	恢复出厂网络设置。当前会话预期会结束。

5. 配置与安全限位

连接后用 get_configurator() 获取配置服务。在 Robot 上它针对 0 号机械臂。这些限位是软件保护。请与机械挡块、控制器硬件限位和单元安全设计一起检查。设置调用返回 RobotResult，并等待确认。

调用	参数 / 结果	用途 / 注意
get_configurator()	返回 Configurator。	对当前已连接会话有效。0 号机械臂。arm(i).get_configurator() 针对该臂。
set_tool_offset(offset) / get_tool_offset()	offset：Position (x, y, z)，不是完整位姿。	工具坐标系运动前设置并读回。改变它会改变工具目标的含义。
set_arm_pos_limits(limit)	逐关节位置软件限位。长度跟随关节数。	确认机器人可达范围后再设置。超过该限位的指令会被保持在限位上。
set_arm_vel_limits(limit)	逐关节速度软件限位。	确认机器人速度能力后再设置。
get_arm_pos_limits() / get_arm_vel_limits()	当前软件限位。	调试时以及任务前读回。
get_arm_max_pos()	硬件最大位置。	越过该限位会使机器人故障。请在机器人客户端恢复。
get_arm_max_vel()	硬件最大速度。	硬件允许的最高速度。

调用	参数 / 结果	用途 / 注意
get_arm_max_pos_jump() / get_arm_max_vel_jump()	位置和速度的防跳变阈值。	大于阈值的台阶会被限制在跳变限位上。
get_arm_max_pos_follow_error()	位置跟随误差阈值。	越过该阈值会使机器人故障。
get_arm_max_vel_follow_error() / get_arm_max_tor_follow_error()	速度和力矩跟随误差阈值。	用于监视。
set/get_gripper_pos_limits(), get_gripper_max_pos()	夹爪软件限位和硬件最大值。	这些是安全限位，不是 move_gripper_pos 的 [0, 1] 单位。

6. 运动学与坐标系变换

Kinematics 是单独的类，不发送运动指令。它需要包含 arm_v2.urdf 和 meshes/ 的配置目录。关节向量单位为度，并且必须匹配该运动学对象的关节数。在线 Robot 类不加载这个目录。

> **本发行包** — 本发行包不包含运动学库和配置目录。

调用	输入 / 输出	用途
Kinematics(config_prefix_path, model_id="")	配置目录，可选模型标识。	加载 URDF 模型。默认模型为 spark2_v2。
forward_kinematics(arm_pos)	输入：关节位置，单位为度。输出：末端位姿。	由已知关节状态求位姿。
inverse_kinematics(ee_pose, arm_pos)	输入：末端位姿和种子关节向量。输出：RobotResult 和求解关节。	仅在结果成功时使用求解出的关节。
ee_to_tool_pose(ee_pose, tool_offset)	末端位姿加平移工具偏移。	在工具坐标系中规划或记录。
tool_to_ee_pose(tool_pose, tool_offset)	工具位姿加平移工具偏移。	把工具目标表示到末端坐标系。
euler_to_quaternion(euler)	内旋 XYZ 欧拉角转为四元数。	构造 Pose 的姿态。
quaternion_to_euler(quat)	四元数转为内旋 XYZ 欧拉角。	显示或记录姿态。

> **逆解** — 逆解只是运动学结果。把它交给运动指令之前，请检查关节限位、碰撞、奇异点裕度和接近路径。

附录 A. 接口索引与类型

A.1 接口分组

分组	内容
生命周期	Robot、enumerate_robots、connect、disconnect、stop、enable_arm_joint、arm、dual_arm、release_control、stop_motion、trigger_safeguard_stop
关节运动	move_pos、move_pos_path、move_vel、go_home、fold_arm、MotionOptions、wait_until_idle、move_and_wait，以及对应的等待调用
笛卡尔	Robot、Arm 和 DualArm 上的 move_ee_point、move_ee_line、move_tool_point、move_tool_line 及对应路径调用
交互	move_gripper_pos、move_gripper_pos_and_wait、示教、stop_recording、save_trajectory、回放、反馈、状态、事件、连接状态、网络配置
配置	get_configurator、工具偏移、臂和夹爪限位、硬件最大值、跳变阈值和跟随误差阈值
运动学	正解、逆解、欧拉角与四元数、末端与工具变换。本发行包不包含。

A.2 类型

类型	含义
RtConnectionInfo	只有 server_ip。地址为空时自动发现。RtConnectionInfo.to(ip) 指定已知主机。
DiscoveredRobot	enumerate_robots() 的一行：服务器地址和机器人名称。传给 connect()。
MotionOptions	speed_limited(percent) 或 timed(seconds)。百分比为 (0, 100] 内的整数。定时的秒数必须大于 0。
JointValues / JointMask	运行时长度的臂或夹爪向量和使能掩码。连接后按关节数构造。

类型	含义
JointState	位置、速度或力矩反馈中的臂向量加夹爪向量。
Pose / Position / Quaternion	笛卡尔位姿。工具偏移只用 Position。
RobotResult	ok、code、message。下一条指令前检查 ok，或调用 raise_for_error()。
SmoothingMethod	LINEAR、COS、CUBIC、QUINTIC、QUINTIC_PATH、LOW_PASS_FILTER、NONE。路径调用选择 QUINTIC_PATH。普通位置运动上的低通设置按有限余弦曲线执行。
RobotState	STARTUP、IDLE、MOVING、SETTLING、ERROR、RECOVERY、SHUTDOWN。
PlanResult	SUCCESS、POSE_NOT_REACHABLE、LINEAR_PATH_FAILED、NONE。NONE 不是运动失败。
SystemStatus	robot_state、plan_result，以及机器人和逐关节诊断标志。
ConnectionStatus	connected、telemetry_timed_out、receive_age_ms、receive_timeout_ms。超时为 3000 毫秒，并保持锁存直到重新连接。